Documentazione del Progetto -

Inventory Manager

Informazioni Personali

Nome e Cognome: **Franks Toscani Koudja Gochui**
E-mail: **franks.koudjagochui@studio.unibo.it**
Numero di Matricola: **0000937188**

---

1. Introduzione

Inventory Manager è un'applicazione mobile sviluppata per la gestione di piccoli inventari. Permette di aggiungere, modificare, eliminare e visualizzare articoli, oltre a esportare e analizzare i dati. Il progetto è stato realizzato interamente in ambiente nativo Android Studio seguendo la metodologia **Agile** e implementando l'architettura **MVVM**(Model-View-ViewModel) per garantire una struttura modulare e facilmente manutenibile.

Tecnologie utilizzate:

- **Linguaggi:** Java, XML
- **Database:** Room Database (SQLite)
- **Grafici:** MPAndroidChart
- **Esportazione:** CSV
- **Scanner:** Libreria ZXing (codici a barre)



2. □ Analisi dei Requisiti

Requisiti Funzionali

- Aggiunta, modifica e cancellazione degli articoli.
- Ricerca avanzata degli articoli.
- Scansione dei codici a barre/Qr formato EAN-13 (European article number).
- Esportazione dei dati in formato **CSV**.
- Visualizzazione delle statistiche dell'inventario su un grafico a barra.
- Notifiche dopo l'esportazione dei dati.

Requisiti Non Funzionali

- Interfaccia utente intuitiva e responsive.
- Prestazioni ottimizzate per dispositivi mobili.
- Compatibilità con diverse versioni di Android.

3. □ Architettura del Sistema

L'app utilizza l'architettura **MVVM (Model-View-ViewModel)** per garantire una chiara separazione dei componenti e facilitare la manutenzione del codice.

Architettura MVVM

- **Model:** `InventoryItem` e `InventoryDao`
- **View:** `Fragment` e `Activity`
- **ViewModel:** Connette la logica al database e alla UI.

Struttura dei package:

- `data.model`: Definizione del modello dati (`InventoryItem`).
- `data.database`: Gestione del database Room (`InventoryDao`, `InventoryDatabase`).
- `ui.list`: Visualizzazione e gestione della lista degli articoli.
- `ui.additem`: Aggiunta di nuovi articoli.
- `ui.analytics`: Visualizzazione di grafici e statistiche.
- `ui.barcode`: Scansione dei codici a barre.
- `utils`: Utility per esportazione dati e notifiche.

4. □ Funzionalità Implementate

- **Gestione dell'Inventario:**
  - Creazione, lettura, aggiornamento e cancellazione degli articoli.
- **Persistenza dei dati:**
  - Room Database.
- **Ricerca Avanzata degli Articoli:**
  - Ricerca per nome, categoria ed EAN.
- **Scanner di Codici a Barre (EAN-13):**
  - Acquisizione automatica dei dati di un prodotto tramite scanner.
- **Esportazione dei Dati:**
  - In formato **CSV**.
- **Statistiche e Grafici:**
  - Visualizzazione di un grafico a barre per analizzare la distribuzione degli articoli in base al nome e categorie.
- **Notifiche Personalizzate:**
  - Notifiche toast per favorire una buona esperienza utente.
  - Notifica con suono personalizzato dopo l'esportazione dei dati.

5. □ Roadmap e Metodologia Agile

Il progetto è stato gestito tramite la metodologia **Agile**, suddiviso in sprint:

- **Sprint-1:** Implementazione del database e operazioni CRUD.
- **Sprint-2:** Modifica e cancellazione degli articoli.
- **Sprint-3:** Integrazione dello scanner di codici a barre.
- **Sprint-4:** Esportazione dei dati (CSV) e ricerca avanzata.
- **Sprint-5:** Miglioramento dell'interfaccia utente, notifiche personalizzate, e ottimizzazione dell'esperienza utente.
- **Sprint-6:** Test, Documentazione e consegna.

  link to repository.

6. Dettaglio delle Classi Java

- **InventoryItem.java**

La classe `InventoryItem` rappresenta il **modello dati** degli articoli gestiti nell'inventario dell'applicazione. Ogni oggetto di questa classe corrisponde a un articolo che viene memorizzato nel database SQLite utilizzando **Room**.

  - Campi:
    - id: identificativo univoco.
    - name: nome del prodotto.
    - quantity: quantità disponibile.
    - category: categoria di appartenenza.
    - eanCode: codice EAN (long).

- **InventoryDao.java**

La classe `InventoryDao` è un interfaccia che definisce le operazioni di accesso e gestione dei dati per la tabella `inventory_items` nel database Room. È annotata con `@Dao` (Data Access Object), che indica a Room che questa interfaccia contiene i metodi per interagire con il database.

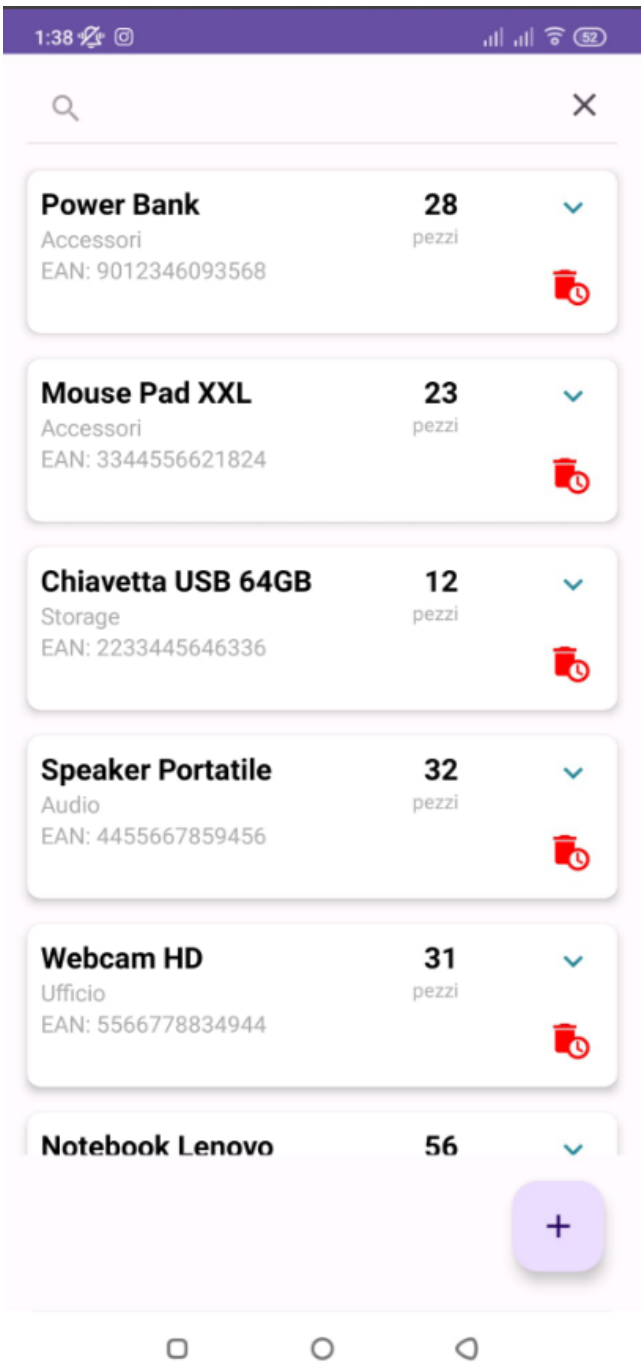
  - Funzionalità:
    - Inserimento, aggiornamento, eliminazione e recupero degli articoli.
    - Metodo per l'esportazione in CSV.

- **InventoryDatabase.java**

La classe `InventoryDatabase` gestisce la **creazione**, **gestione** e **accesso** al database locale dell'applicazione, utilizzando la libreria **Room** di Android. Questa classe definisce la struttura del database, il DAO (Data Access Object) per le operazioni sui dati e gestisce eventuali migrazioni tra versioni.

  - Funzionalità:
    - Connessione tra DAO e database.
    - Gestione delle migrazioni.

- **MainActivity.java:**
    - Ruolo: Gestisce la schermata principale.
    - Funzionalità: Carica il ListFragment che mostra la lista degli articoli.
      stata sostituita in parte dalla WelcomeActivity per una migliore navigazione durante lo sviluppo..



- **WelcomeActivity.java**

La `WelcomeActivity` rappresenta la schermata principale dell'applicazione *Inventory Manager*. è il punto di accesso alle principali funzionalità dell'app, offrendo un'interfaccia semplice e intuitiva tramite l'uso di **CardView**. Gestisce la navigazione tra le funzionalità più importanti dell'app, come la gestione dell'inventario, l'esportazione dei dati e la visualizzazione delle analisi, mantenendo le operazioni efficaci e sicure.



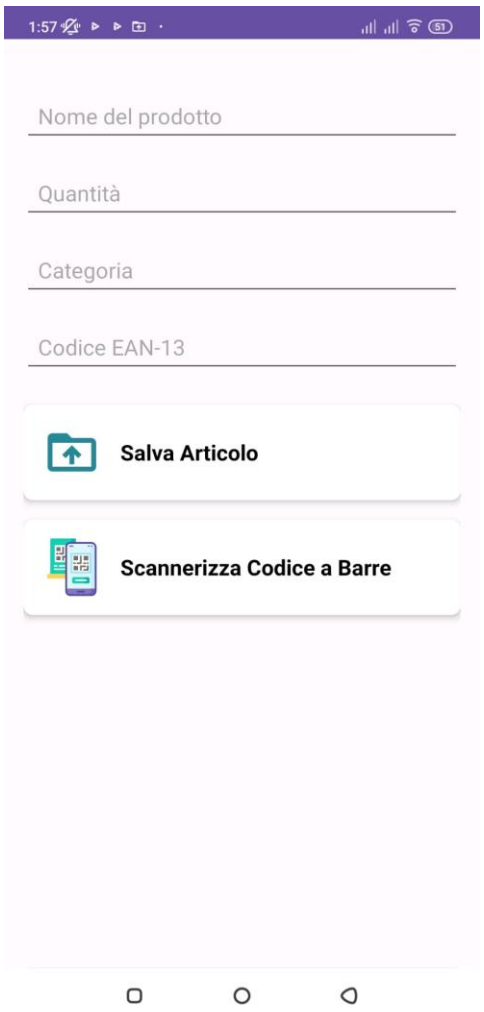
- **ListFragment.java**

- Ruolo: Visualizza la lista di articoli.
- Funzionalità:
  - Mostra gli articoli in un RecyclerView.
  - Supporta la ricerca avanzata degli articoli.
  - Consente la modifica rapida tramite popup e la cancellazione.

Il codice di questa classe gestisce la visualizzazione e l'interazione con la lista degli articoli dell'inventario. È responsabile di mostrare, cercare, aggiungere e cancellare articoli nella lista.

- **AddItemFragment.java**

  - Ruolo: Permette l’aggiunta di nuovi articoli.
  - Funzionalità:
    - Form con campi per Nome, Quantità, Categoria, EAN Code.
    - Pulsante per attivare lo scanner dei codici a barre.
    - Salvataggio degli articoli nel database.



- **EditItemFragment.java**

- Ruolo: Modifica degli articoli esistenti.
- Funzionalità:
  - Popola automaticamente i dati dell'articolo da modificare.
  - Consente la modifica dei dati e il salvataggio nel database.
  - È stato implementato con una popup dialog per una modifica più veloce.



- **ItemAdapter.java**
  - Ruolo: Fornisce l'implementazione di un **Adapter** per la `RecyclerView` nell'app
  - Funzionalità:
    - Visualizza ogni articolo in una CardView.
    - Implementa i pulsanti per modifica e cancellazione.
    - Aggiunge effetti di animazione alle card.

In sintesi, questa classe gestisce in modo completo la **visualizzazione dinamica**, la **modifica** e la **rimozione** degli articoli dell'inventario con un'interfaccia utente moderna e qualche animazione fluida.

- **BarcodeScannerFragment.java**
  - Ruolo: Scannerizzazione dei codici a barre.
  - Funzionalità:
    - Utilizza la libreria ZXing per leggere codici EAN-13.
    - Interpreta i dati scansionati e li passa all'AddItemFragment.

- **AnalyticsFragment.java**

La classe si occupa di visualizzare un grafico a barre che rappresenta graficamente la quantità degli articoli dell'inventario suddivisi per categoria. Questa visualizzazione aiuta l'utente a comprendere meglio la distribuzione e lo stato dell'inventario**.**

    - Ruolo: Analisi dei dati tramite grafici.
    - Funzionalità:
      - Utilizza la libreria MPAndroidChart.
      - Visualizza grafici a barre e grafici per analizzare la distribuzione degli articoli.

Il **grafico a barre** fornisce una panoramica chiara e intuitiva dell'inventario, migliorando l'esperienza utente e supportando decisioni rapide sulla gestione degli articoli.

- **CsvExporter.java**

La classe `CsvExporter` gestisce l'esportazione dei dati dell'inventario in formato CSV. Questo è utile per creare copie di backup, condividere i dati o elaborarli con altri strumenti come Excel o Google Sheets**.**

- **NotificationUtils.java**
    - Ruolo: Gestione delle notifiche.
    - Funzionalità:
      - Crea notifiche personalizzate con suono.
      - Avvisa l'utente al termine dell'esportazione.

- **`ListViewModel`.java**

La classe `ListViewModel` segue il pattern MVVM (Model-View-ViewModel) ed è utilizzata per gestire i dati della lista degli articoli in modo reattivo, separando la logica di business dalla UI. Questo approccio aiuta a mantenere il codice più pulito, modulare e testabile.

- **`ListViewModelFactory.java`**

La classe `ListViewModelFactory` serve a creare istanze di `ListViewModel` in modo sicuro.

- **`AddItemViewModel`.java**

La classe `AddItemViewModel` ha il compito di gestire la logica di inserimento degli articoli (`InventoryItem`) nel database, separando questa logica dalla UI.

6. □□ Conclusioni e Sviluppi Futuri

Il progetto **Inventory Manager** ha raggiunto tutti gli obiettivi prefissati.

Possibili sviluppi futuri:

- Implementazione della gestione di inventari multipli.
- Integrazione di un sistema di login per utenti.
- Sincronizzazione dei dati su cloud.
- Implementazione di un sistema di backup automatico.

---

7. Riferimenti

- Documentazione ufficiale Android
- Libreria ZXing
- Room Database

8. Dati per il Test









SCAN ME


SCAN ME


SCAN ME


SCAN ME


SCAN ME


SCAN ME


SCAN ME